

UNIT V

Dr. K. K. THYAGHARAJAN
Professor & Dean (Research)
R.M.D. Engineering College

Scan the QR code and it will provide the link to all videos. It is a trusted link and safe.



<https://www.youtube.com/c/KKTVirtualLearning>

1. What is a Decision Tree?

A decision tree is a supervised learning algorithms used for both **classification and regression** tasks.

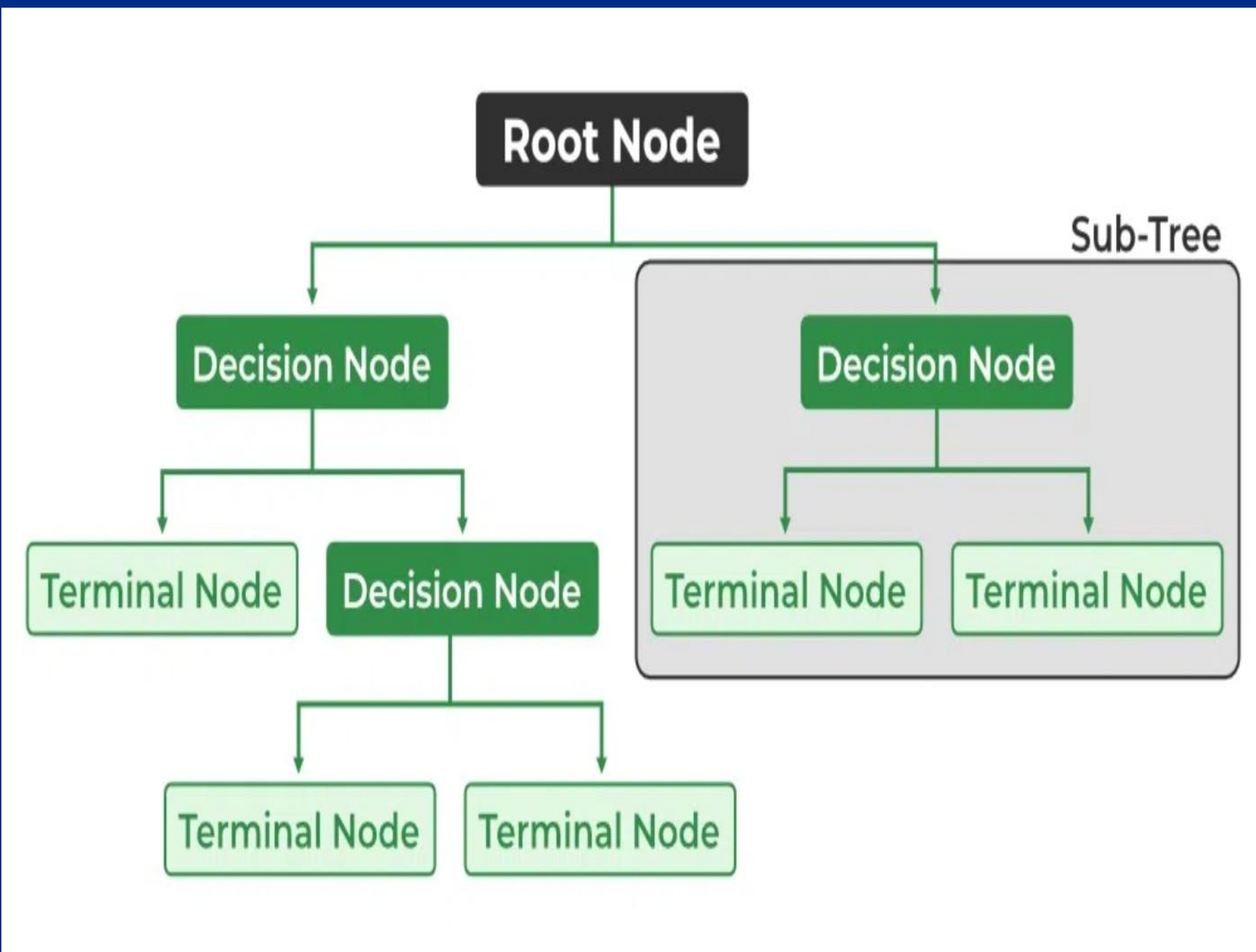
It builds a **flowchart-like tree structure** where each internal node denotes a test on an attribute, each branch represents an outcome of the test, and each leaf node (terminal node) holds a class label.

It is constructed by recursively splitting the training data into subsets based on the values of the attributes until a stopping criterion is met, such as the maximum depth of the tree or the minimum number of samples required to split a node.

During training, the Decision Tree algorithm selects the best attribute to split the data based on a metric such as entropy or Gini impurity, which measures the level of impurity or randomness in the subsets.

The goal is to find the attribute that maximizes the information gain or the reduction in impurity after the split.

1. What is a Decision Tree? Contd. ...



Root Node: It is the topmost node in the tree, which represents the complete dataset. It is the starting point of the decision-making process.

Decision/Internal Node: A node that symbolizes a choice regarding an input feature. Branching off of internal nodes connects them to leaf nodes or other internal nodes.

Leaf/Terminal Node: A node without any child nodes that indicates a class label or a numerical value.

Parent Node: The node that divides into one or more child nodes.

Child Node: The nodes that emerge when a parent node is split.

Splitting: Splitting a node into two or more sub-nodes using a **split criterion and a selected feature**.

Branch/Sub-Tree: A subsection of the decision tree starts at an internal node and ends at the leaf nodes.

1. What is a Decision Tree? Contd. ...

Advantages of the Decision Tree:

It is simple to understand as it follows the same process which a human follow while making any decision in real-life.

It can be very useful for solving decision-related problems.

It helps to think about all the possible outcomes for a problem.

There is less requirement of data cleaning compared to other algorithms.

Disadvantages of the Decision Tree:

In Decision Tree the major challenge is to identification of the attribute for the node in each level.

The decision tree contains lots of layers, which makes it complex.

Trees that are too deep can lead to models that are too detailed and don't generalize on new data. This is called overfitting and it can be resolved using the Random Forest algorithm.

For more class labels, the computational complexity of the decision tree may increase.

Trees that are too shallow might lead to overly simple models that can't fit the data.

Pruning the branches or stopping the tree early are used to avoid overfitting of data to the tree

2. Construction of Decision Tree

The construction of a decision tree classifier **does not require any domain knowledge or parameter setting** and therefore is appropriate for exploratory knowledge discovery.

Decision trees **can handle high-dimensional data**.

Assumptions Made:

We consider the whole training set as the root node

The decision tree method is ordinarily employed for categorizing Boolean examples, such as yes or no i.e. the target function has 'yes' or 'no'. If the values are continuous then they are discretized prior to building the model.

Advanced versions of the algorithm can accommodate attributes with continuous numerical values, such as representing temperature with a numerical scale

In decision tree learning, instances are represented by attribute-value pairs, e.g. (temp, hot)

Each attribute will have a finite set of distinct values, like hot, mild, or cold.

Tree works on the Sum of Product form which is also known as Disjunctive Normal Form.

2. Construction of Decision Tree Contd. ...

Step-1: Begin the tree with the root node, which contains the complete dataset.

Step-2: Find the best attribute in the dataset using Attribute Selection Measure (ASM). e.g. entropy, information gain or Gini Index

Step-3: Split the data set S into subsets using the attribute data with a threshold.

Step-4: Generate the decision tree node with the best attribute that has **maximum information gain or minimum Gini Index**

Step-5: Recursively make new decision trees using the subsets of the dataset created in step -3.

Continue this process until a stage is reached where you cannot further classify the nodes and called the final node as a leaf node Classification and Regression Tree algorithm.

Note: No root-to-leaf path should contain the same discrete attribute twice

2. Construction of Decision Tree Contd. ...

The border cases:

If all positive or all negative training instances remain, label that node —yes || or —no || accordingly

If no attributes remain, label with a majority vote of training instances left at that node

If no instances remain, label with a majority vote of the parent's training instances Example:

2. Construction of Decision Tree Contd. ...

The most notable types of decision tree algorithms are:-

Iterative Dichotomiser 3 (ID3): This algorithm uses Information Gain to decide which attribute is to be used to classify the current subset of the data. For each level of the tree, information gain is calculated for the remaining data recursively.

C4.5: This algorithm is the successor of the ID3 algorithm. This algorithm uses either Information gain or Gain ratio to decide upon the classifying attribute. It is a direct improvement from the ID3 algorithm as it can handle both continuous and missing attribute values.

Classification and Regression Tree (CART): It is a dynamic learning algorithm which can produce a regression tree as well as a classification tree depending upon the dependent variable.

3. Construction of Decision Tree – Method 1

Attribute Selection for Splitting a parent Node Using Entropy or Information Gain

When we use a node in a decision tree to partition the training instances into smaller subsets the entropy changes. Information gain is a measure of this change in entropy.

Entropy: Entropy is the measure of the **degree of randomness or uncertainty** in the dataset. In the case of classifications, It measures the randomness based on the distribution of class labels in the dataset.

Let S be the dataset that contains K number of classes and 'n' number of data. So the root node has n data, then the entropy of the root (i^{th}) node can be defined as:

$$H_i = - \sum_{k=1}^K p_i(k) \log_2(p_i(k))$$

$$\text{where } p_i(k) = \frac{\text{Number of data in class } k \text{ of node } i}{\text{Total number of data in node } i}$$

i is the specific node

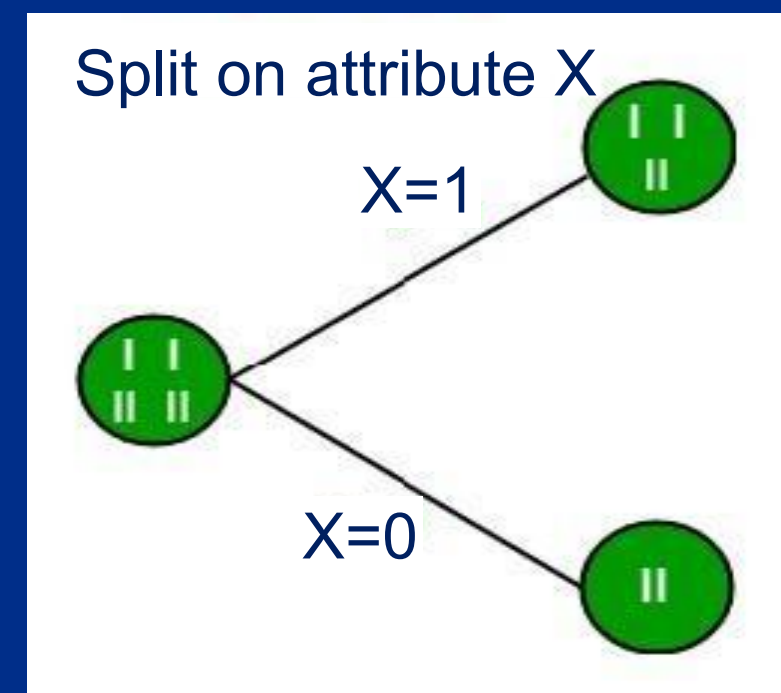
K is number of classes available in that specific node ' i '

k indicates a specific class

For example, you consider the root node (i)

The number of classes (K) in that specific node ' i ' is 2 (i.e. I & II)

In the first class (I) i.e. for $k=1$ there are two data and in the second class ($k=2$) there are 2 data. So, the total number of data is 4



3. Construction of Decision Tree – Method 1 Contd. ...

Information Gain:

In order to evaluate how good a feature is for splitting, the difference in entropy before and after the split is calculated.

Information gain = Entropy of parent node – Sum of weighted average of entropy of children

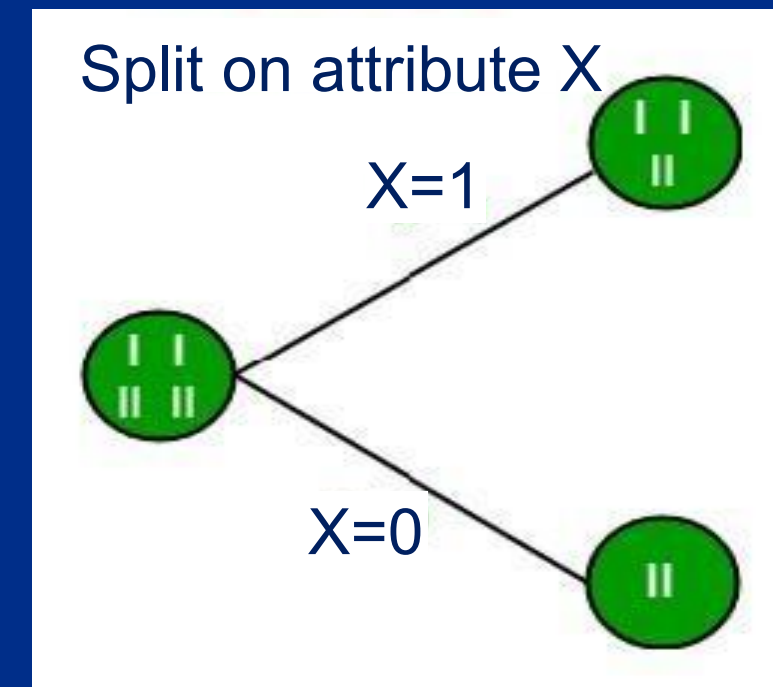
$$\text{Information gain} = E_p - \sum_{i=1}^I \frac{N_i}{N_p} E_i$$

where E_p is the entropy of parent node and

N_i is the number of data in the i^{th} node

N_p is the number data in the parent node

E_i is the entropy of the n th node



Entropy is used to evaluate the quality of a split. **The goal of entropy is to select the attribute that minimizes the entropy of the resulting subsets**

The total entropy of the next level child nodes is 0. In such a case the data in the child node will be completely homogeneous, meaning that each instance (data) in that node perfectly fits to that node or class.

3. Construction of Decision Tree – Method 1 Contd. ...

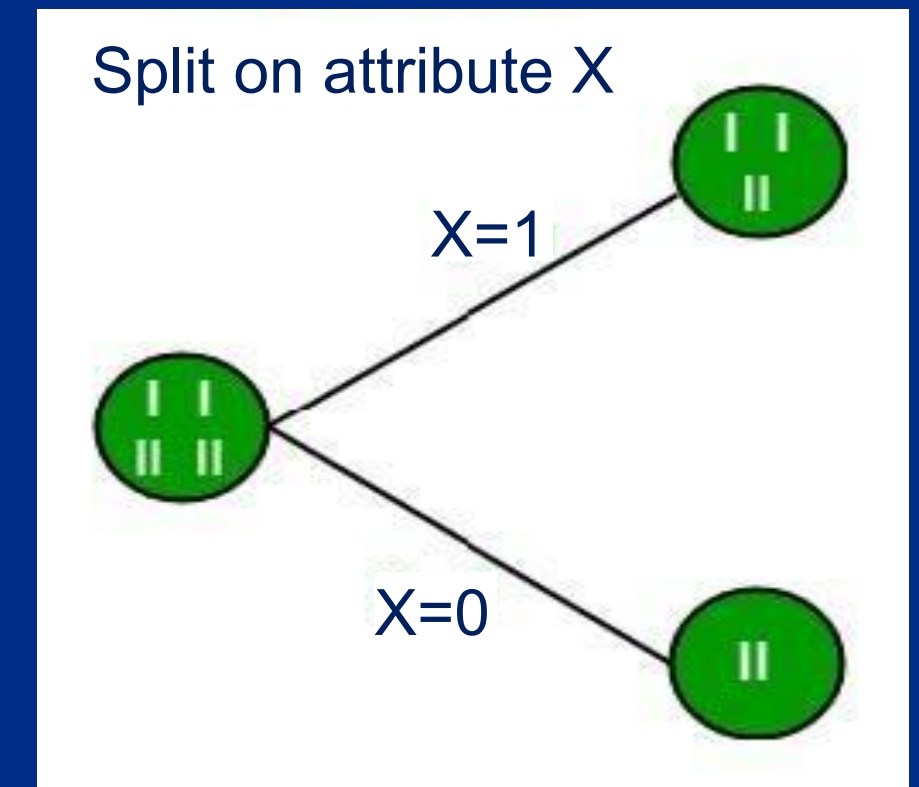
Construct a Decision Tree for the following data using Information gain. Training set: 3 features or attributes (X,Y and Z) and 2 classes (I & II)

X	Y	Z	Class
1	1	1	I
1	1	0	I
0	0	1	II
1	0	0	II

At root node , complete data set is taken. There are two classes, each class has two data

$$E_i = - \sum_{k=1}^K p_i(k) \log_2(p_i(k))$$

$$\text{Information gain} = E_p - \sum_{i=1}^I \frac{N_i}{N_p} E_i$$



1. Entropy for parent (root) node :

K the total number of classes =2 ;
 probability for class I : $p(\text{class I}) = 2/4$
 probability for class II : $p(\text{class II}) = 2/4$

$$\begin{aligned} E_p &= -\frac{2}{4} \log_2 \left(\frac{2}{4} \right) - \frac{2}{4} \log_2 \left(\frac{2}{4} \right) \\ &= -\frac{2}{4} \log_2 (2^{-1}) - \frac{2}{4} \log_2 (2^{-1}) \\ &= -\frac{2}{4} (-1) \log_2 (2) - \frac{2}{4} (-1) \log_2 (2) \\ &= \left(\frac{2}{4} \right) \times (1) + \left(\frac{2}{4} \right) \times (1) = 1 \end{aligned}$$

So, for root node, the entropy is 1

2. Entropy for children:

For the 1st children i.e. X=1, K=2

Probability for class I = 2/3

Probability for class II = 1/3

Entropy for node X=1:

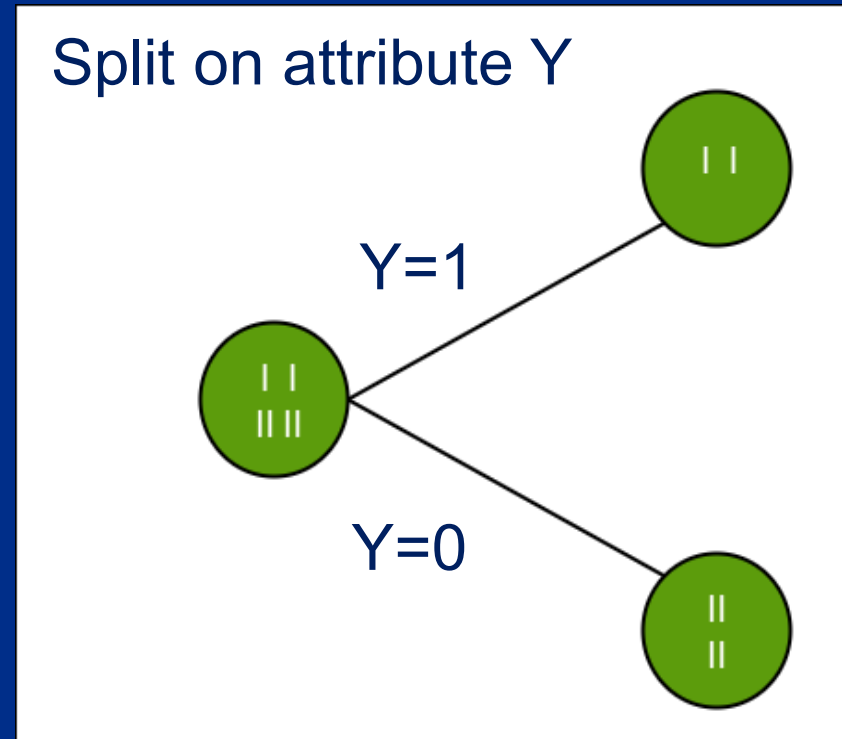
$$\begin{aligned} E_1 &= -\frac{2}{3} \log_2 \left(\frac{2}{3} \right) - \frac{1}{3} \log_2 \left(\frac{1}{3} \right) \\ &= 0.399 + 0.5284 = 0.9183 \end{aligned}$$

$$E_0 = -1 \log_2 (1) = 0$$

$$\begin{aligned} \text{Information Gain} &= \text{parent entropy} - \text{weighted sum of children's entropy} \\ &= 1 - \left\{ \left(\frac{n_1}{n_r} E_1 \right) + \left(\frac{n_0}{n_r} E_0 \right) \right\} \\ &= 1 - \left\{ \left(\frac{3}{4} \times 0.9183 \right) + \left(\frac{1}{4} \times 0 \right) \right\} \\ &= 1 - (0.688725) \\ &= 0.311275 \end{aligned}$$

3. Construction of Decision Tree – Method 1 Contd. ...

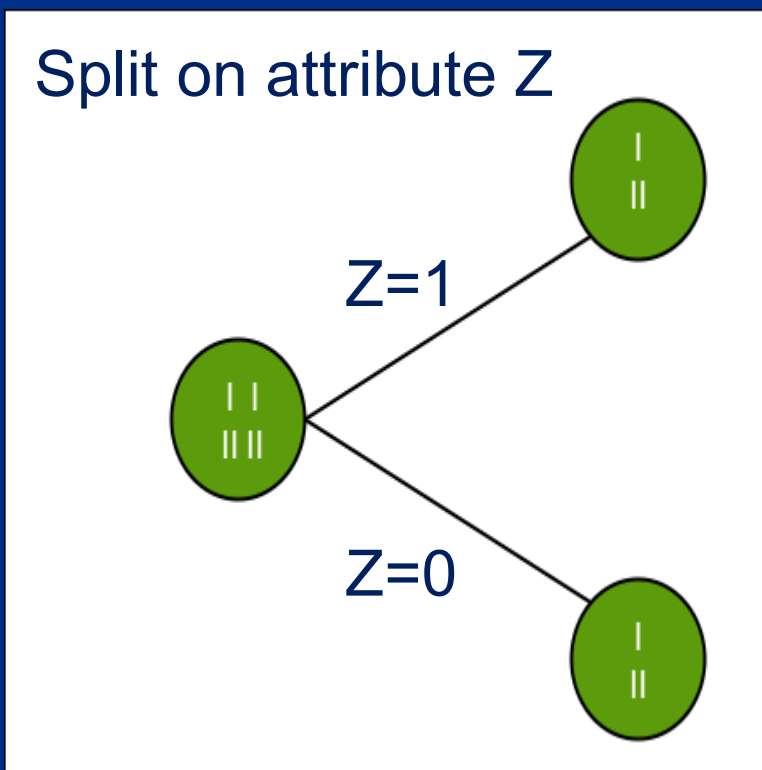
X	Y	Z	Class
1	1	1	I
1	1	0	I
0	0	1	II
1	0	0	II



$$\text{Entropy for node } Y=1 : E_1 = -\frac{2}{2} \log_2 \left(\frac{2}{2} \right) = 0$$

$$E_0 = -\frac{2}{2} \log_2 \left(\frac{2}{2} \right) = 0$$

$$\begin{aligned} \text{Information Gain} &= 1 - \left\{ \left(\frac{n_1}{n_r} E_1 \right) + \left(\frac{n_0}{n_r} E_0 \right) \right\} \\ &= 1 - \left\{ \left(\frac{2}{4} \times 0 \right) + \left(\frac{2}{4} \times 0 \right) \right\} = 1 - (0) = 1 \end{aligned}$$



$$\text{Entropy for node } Z=1: E_1 = -\frac{1}{2} \log_2 \left(\frac{1}{2} \right) - \frac{1}{2} \log_2 \left(\frac{1}{2} \right) = 0.5 + 0.5 = 1.0$$

$$\text{Entropy for node } Z=0: E_0 = -\frac{1}{2} \log_2 \left(\frac{1}{2} \right) - \frac{1}{2} \log_2 \left(\frac{1}{2} \right) = 0.5 + 0.5 = 1.0$$

$$\text{Information Gain} = 1 - \left\{ \left(\frac{2}{4} \times 1 \right) + \left(\frac{2}{4} \times 1 \right) \right\} = 1 - 1 = 0$$

Data will be classified correctly when the **information gain is maximum**.

If the entropy is 0, then the data will be of one class only and further split is not necessary

In the above example, the table contains 4 data and Y attribute can be used to split the data into two groups. **Since the information gain is 1 and the entropy is zero, further split is not necessary.**

4. Construction of Decision Tree Method 2

Attribute Selection for Splitting the a parent Node Using Gini Index

Gini Impurity:

Another measure for purity — or actually impurity — used by the Classification And Regression Tree (CART) algorithm is the Gini index.

Gini Index is a metric to measure how often a randomly chosen element would be incorrectly identified. It means that an attribute with lower Gini index should be selected to split a node.

The Gini Impurity evaluates a score in the range between 0 and 1, where 0 is when all observations belong to one class, and 1 is a random distribution of the elements within classes. In this case, we want to have a Gini index score as low as possible.

$$\text{Gini Impurity of } n^{th} \text{ node } G_n = 1 - \sum_{k=1}^K (p_n(k))^2$$

where K is the number of classes &
 $p_n(k) = \frac{\text{Number of data in class } k \text{ of node } n}{\text{Total number of data in node } n}$

4. Construction of Decision Tree Method 2 Contd. ...

Gini Index:

The Gini index is defined as the weighted sum of the Gini impurity of the different subsets after a split, where each portion is weighted by the ratio of the size of the subset with respect to the size of the parent dataset.

For a dataset with two classes, the range of the Gini index is between 0 and 0.5 and the Gini Index will be 0 if the dataset is pure and 0.5 if the two classes are distributed equally.

If there are N number of nodes in the next lower level, the Gini Index of that level for a specific attribute is given by

$$\text{Gini Index} = \sum_{n=1}^N \frac{D_n}{D_p} G_n$$

where N is the number of nodes

'n' is a specific class

D_n is the number of data in the nth node

D_p is the number data in the parent node

G_n is the Gini Impurity of the nth node

The attribute that has lower Gini Index should be put at higher level

Avoiding Overfitting

Overfitting of the training data on the machine learning model can be reduced by using

1. Pruning
2. Early stopping
3. Tuning hyperparameters

Pruning - In Decision Trees

Decision trees can be exceptionally huge and perplexing, prompting overfitting and decreased generalization capacity. So, pruning can be used to eliminate unnecessary branches and nodes that are not adding value in reducing the classification error but redundant in the decision tree. This results in a smaller and simpler model that is less liable to overfit and hence regularize the model.

The effect of pruning on the model's accuracy should be evaluated using fitting metrics, such as validation accuracy or test accuracy.

Iterative pruning involves pruning the model on various occasions during training, which can prompt improved results than a single pruning toward the finish of training.

Advantages:

Decreases the model size and complexity.

Faster inference with reduced computational cost

Improved generalization capacity and reduced overfitting.

Increased interpretability.

Pruning can be joined with other regularization techniques, such as **L1 and L2 regularization or dropout**, to further develop the model's performance further.

Pruning - In Decision Trees Contd. ...

Disadvantages:

Pruning can sometimes result in a loss of accuracy, especially when a large number of parameters are pruned (over pruning) or when pruning is not done cautiously.

The **pruning rate** determines the proportion of parameters to be pruned. It should be chosen cautiously to adjust the reduction in model size without the loss of accuracy.

Pruning can increase the training time of the model because more iteration may be required for training.

Choosing the right pruning technique may require expertise and experimentation

Pruning - In Decision Trees Contd. ...

There are two types of pruning that may be adopted in Decision Trees

a) Post-Pruning: This is more commonly used and replaces the subtrees when they are deemed unnecessary. Since there was already a completed decision tree, after pruning, we can see the improvements in accuracy with reduced complexity in the pruned tree.

b) Pre-Pruning: This method uses a stopping criterion to prevent the tree from further expansion. An example of such a stopping criterion is the **depth of the tree**, meaning it is predetermined how many consecutive decision nodes are allowed. This process is generally considered efficient because it keeps the tree small from the beginning and does not require the entire dataset. However, it can lead to premature pruning without considering all the information, resulting in the loss of important information. This effect is referred to as the **Horizon Effect**.

Pruning can be done from the bottom of the tree and moving upwards (**Bottom-Up pruning**) or from the root (top) of the tree and moving downwards to the leaf nodes (**Top-Down**). For Top-Down Pruning, there is a risk that subtrees may be pruned prematurely, even if relevant nodes are still below.

Early Stopping:

Another option to avoid **overfitting** is **early stopping**, based on a stopping criterion.

One common stopping criterion is to set the minimum number of samples per node. The branch will stop its growth when a node created is containing fewer or an equal number of data samples as the minimum set number.

A higher value of this minimum number leads to shallower trees, while a smaller value leads to deeper trees.

Hyperparameter & Tuning:

A **hyperparameter** is a parameter whose value cannot be **estimated from data** and it can not be changed during the training process. These hyperparameters are used to improve the learning of the model. The hyperparameters are **external** to the model but they are used to estimate the **model parameters** which are **internal** to the model. Model parameters are learnt from data itself.

The value of the Hyperparameter is selected and set by the machine learning engineer **heuristically** before start training the model.

The training process should help to find out the best / optimal hyperparameters that can used by learning algorithms to provide the best results or predictions.

The prefix "hyper" suggests that the parameters are top-level parameters that are used in controlling the learning process.

They are often **tuned for a given predictive** modeling problem.

The best value of hyperparameter on a given problem is not known. To determine the best value, thumb rules may be used or values used on other issues may be copied, or search for the best value by trial and error.

Model Parameters:

There is always a big confusion between Parameters and hyperparameters. Model parameters or simply parameters are learnt for data itself. They are usually not set manually. These are the part of the model i.e. they are configuration variables that are internal to the model. They are used by the model for making predictions.

Examples:

- i) Weights or Coefficients of independent variables in the Linear regression model.
- ii) Weights or Coefficients of independent variables in SVM
- iii) Weights, and biases of a neural network
- iv) Cluster centroid in clustering.

Types of Hyper parameters:

hyperparameters can be divided into two categories:

A) Hyperparameters for Optimization: The process of selecting the best hyperparameters to use is known as hyperparameter **tuning**, and the tuning process is also known as hyperparameter optimization.

Examples: Learning Rate, Batch size, Number of Epochs

B) **Hyperparameters for Specific Models:** Hyperparameters that are involved in the structure of the model are known as hyperparameters for specific models.

Examples: Number of Hidden Units, Number of Layers

Examples of Hyper parameters:

i) Learning Rate:

The learning rate is the hyperparameter in optimization algorithms that controls how much the model needs to change in response to the estimated error for each time when the model's weights are updated.

It is one of the crucial parameters while building a neural network, and also it determines the frequency of cross-checking with model parameters.

Selecting the optimized learning rate is a challenging task because if the learning rate is very less, then it may slow down the training process.

On the other hand, if the learning rate is too large, then it may not optimize the model properly.

ii) Batch Size:

To enhance the speed of the learning process, the training set is divided into different subsets, which are known as a batch.

iii) Number of Epochs:

An epoch can be defined as the complete cycle for training the machine learning model. Epoch represents an iterative learning process.

The number of epochs required varies from model to model, and models are created by training with more than one epoch.

To determine the right number of epochs, a validation error is taken into account. The number of epochs is increased until there is a reduction in a validation error.

If there is no improvement in reduction in the error for the consecutive epochs, then it indicates you have to stop increasing the number of epochs.

Challenges in Hyperparameter Tuning

Dealing with High-Dimensional Hyperparameter Spaces: Efficient Exploration and Optimization

Handling Expensive Function Evaluations: Balancing Computational Efficiency and Accuracy

Incorporating Domain Knowledge: Utilizing Prior Information for Informed Tuning

Developing Adaptive Hyperparameter Tuning Methods: Adjusting Parameters During Training

Advantages of Hyperparameter tuning:

Improved model performance

Reduced overfitting and underfitting

Enhanced model generalizability

Optimized resource utilization

Improved model interpretability

Disadvantages of Hyperparameter tuning:

Computational cost

Time-consuming process

Risk of overfitting

No guarantee of optimal performance

Requires expertise

Ensemble Learning

Ensemble learning is a machine learning technique that combines predictions or outputs of multiple models for producing precise prediction.

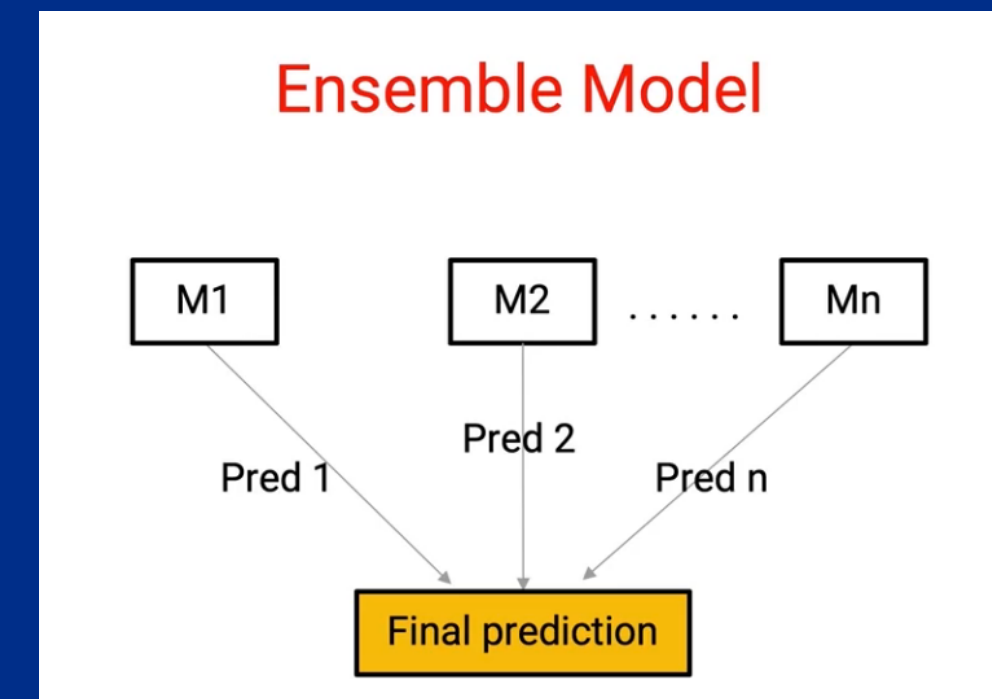
It aims to reduce the errors or biases that may exist in individual models and enhances the accuracy and resilience in forecasting

By considering multiple perspectives and utilizing the strengths of different models, ensemble learning improves the overall performance of the learning system.

This approach not only enhances accuracy but also provides resilience against uncertainties in the data.

The following ensemble techniques are used:

1. Max Voting
2. Averaging
3. Weighted Averaging
4. Rank Averaging



1. Max Voting (Majority Voting) Ensemble Method:

The max voting method is generally used for classification problems. In this technique, multiple models are used to make predictions for each data point. The predictions by each model are considered as a 'vote'. The predictions which we get from the majority of the models are used as the final prediction.

This method assumes that the all models in the ensemble are equally skillful and contribute same proportions (i.e. same weightage for each model) for predictions made by the ensemble.

For example, when 5 persons rate a movie on a 5 point scale (similar to 5 models giving their predictions), we'll assume three of them rated it as 4. while two of them gave it a rating of 5. Since the majority gave a rating of 4, the final rating will be taken as 4. You can consider this as taking **the mode** of all the predictions.

Person 1	Person 2	Person 3	Person 4	Person 5	Final rating
5	4	5	4	4	4

2. Averaging Ensemble Method:

In this method, we take an average of predictions from all the models and use it to make the final prediction. Averaging can be used for making predictions in regression problems or while calculating probabilities for classification problems.

For example, in the below case, the averaging method would take the average of all the values.

i.e. $(5+4+5+4+4)/5 = 4.4$

3. Weighted Average or Weighted Sum Ensemble Method:

Weighted average ensembles assume that some models in the ensemble have more skill than others and give them more contribution when making predictions. So, models are assigned different weights defining the importance of each model for prediction. For instance, in the previous example, if two persons are **critics**, while others have **no prior experience** in this field, then the answers by these two friends are given more importance as compared to the other people.

	Person 1	Person 2	Person 3	Person 4	Person 5	Weighted Average
weight	0.23	0.23	0.18	0.18	0.18	
rating	5	4	5	4	4	4.41

$$\text{Weighted Average} = [(5*0.23) + (4*0.23) + (5*0.18) + (4*0.18) + (4*0.18)] = 4.41$$

4. Rank Averaging: This method uses a normalized validation score of all the models to rank a model and decide the weight for that model, and this can help to improve generalization. For example, you can use R-square (goodness of fit) value of the validation set to assign the rank or weight for a model.

	R-square on validation	Rank
M1 →	0.6	2
M2 →	0.4	1
M3 →	0.7	3

In this diagram M1, M2 and M3 are different machine learning models. The R-square value of Model 2 is the lowest i.e. it does not produce a good fit, it is worst performing. So it is assigned the lowest value 1 (Rank 1). The next lower value is for M1, so its rank is 2. M3 is assigned Rank 3.

	R-square on validation	Weights
M1 →	0.6	0.33
M2 →	0.4	0.16
M3 →	0.7	0.50

In the case of Model 2, we'll divide 1 (rank) by the sum of $1+2+3 = 6$. So the weight for Model 2 comes down to $1/6 = 0.16$.

Similarly for model 1, the weight is $2/6$ i.e. 0.33 and for model 3, the weight is $3/6=0.5$

These weights are used as we did in Weighted Average method

Advanced Ensemble Learning Contd. ...

3. Bagging (Bootstrap Aggregating): Bootstrapping is a sampling technique in which subsets (bags) of the original dataset are created. The total size of all the subsets is the same as the size of the original set.

Each subset is trained by a different base model (weak model).

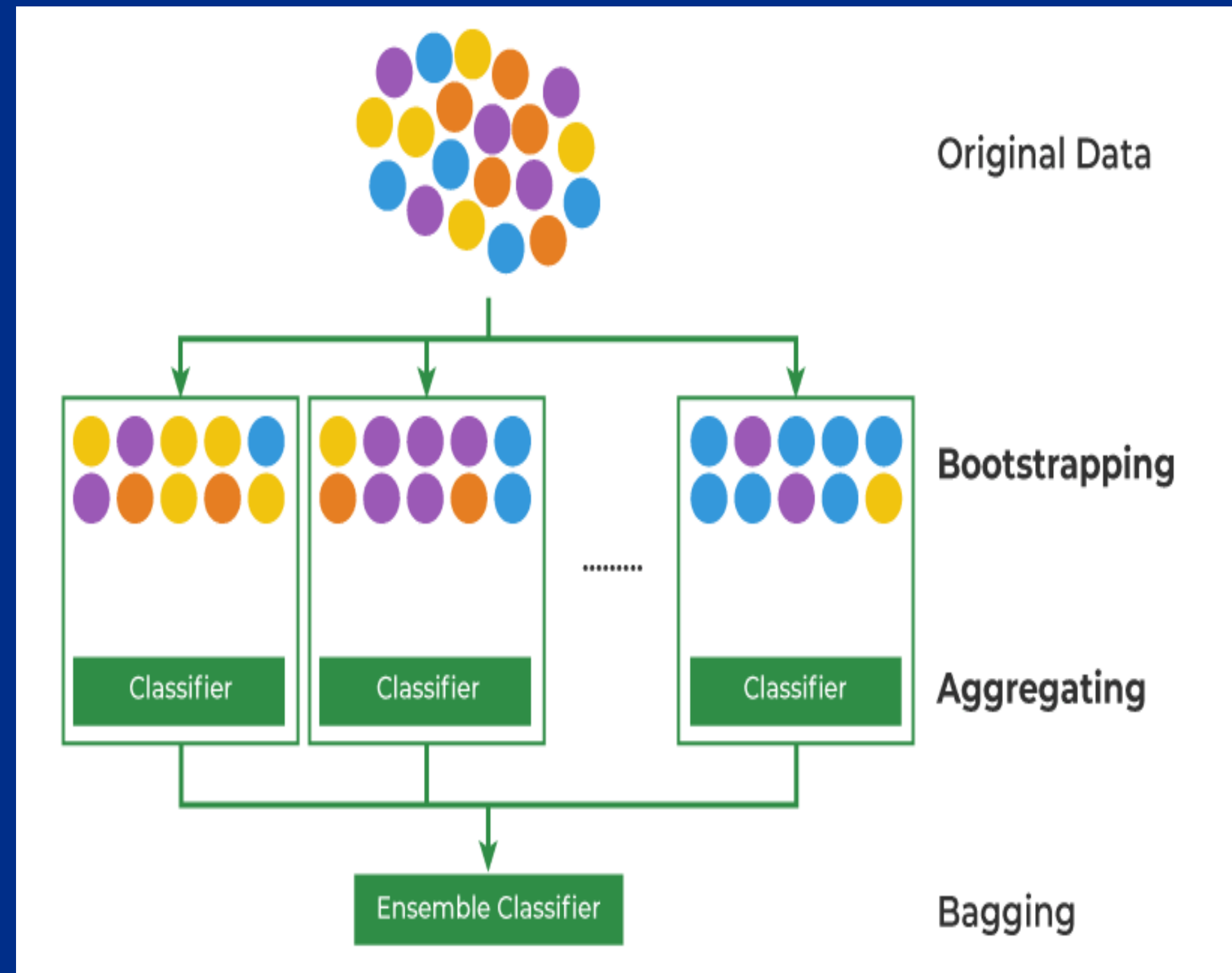
While predicting, the models are run in parallel and are independent of each other.

The predicted class label for the given instance is chosen based on the majority voting (aggregation of results from all models).

This method reduces the risks of overfitting and improves the accuracy of the model

Bagging algorithms:

- a) Bagging meta-estimator
- b) Random forest



3a) Bagging meta-estimator: Bagging meta-estimator is an ensembling algorithm that can be used for both classification (Bagging Classifier) and regression (Bagging Regressor) problems

Implementation Steps:

- i) Random subsets are created from the original dataset (Bootstrapping).
- ii) The subset of the dataset includes all features.
- iii) A user-specified base estimator is fitted on each of these smaller sets.
- iv) Predictions from each model are combined to get the final result.

Advantages of Bagging Classifier:

Improved Predictive Performance: Bagging Classifier often outperforms single classifiers by reducing overfitting and increasing predictive accuracy. By combining multiple base models, it can better generalize to unseen data.

Robustness: Bagging reduces the impact of outliers and noise in the data by aggregating predictions from multiple models. This enhances the overall stability and robustness of the model.

Reduced Variance: Since each base model is trained on different subsets of the data, the aggregated model's variance is significantly reduced compared to an individual model.

Parallelization: Bagging allows for parallel processing, as each base model can be trained independently. This makes it computationally efficient, especially for large datasets.

Flexibility: Bagging Classifier is a versatile technique that can be applied to a wide range of machine learning algorithms, including decision trees, random forests, and support vector machines.

Disadvantages of Bagging Classifier:

Loss of Interpretability: Bagging involves aggregating predictions from multiple models, making it harder to interpret the individual contributions of each base model. This can limit the ability to derive precise business insights from the ensemble.

Computationally Expensive: As the number of iterations (bootstrap samples) increases, the computational cost of bagging also grows. This makes it less suitable for real-time applications where efficiency and speed are crucial. Efficient parallel processing or distributed systems are often required for handling large datasets and numerous iterations.

Less Flexible: Bagging is most effective when applied to algorithms that are less stable or more prone to overfitting. Algorithms that are already stable or exhibit low bias might not benefit significantly from bagging, as there is less variance to be reduced within the ensemble.

Applications of Bagging Classifier:

Fraud Detection: Bagging Classifier can be used to detect fraudulent transactions by aggregating predictions from multiple fraud detection models.

Spam filtering: Bagging classifier can be used to filter spam emails by aggregating predictions from multiple spam filters trained on different subsets of the spam emails.

Credit scoring: Bagging classifier can be used to improve the accuracy of credit scoring models by combining the predictions of multiple models trained on different subsets of the credit data.

Image Classification: Bagging classifier can be used to improve the accuracy of image classification tasks by combining the predictions of multiple classifiers trained on different subsets of the training images.

Natural language processing: In NLP tasks, the bagging classifier can combine predictions from multiple language models to achieve better text classification results.

Random Forest

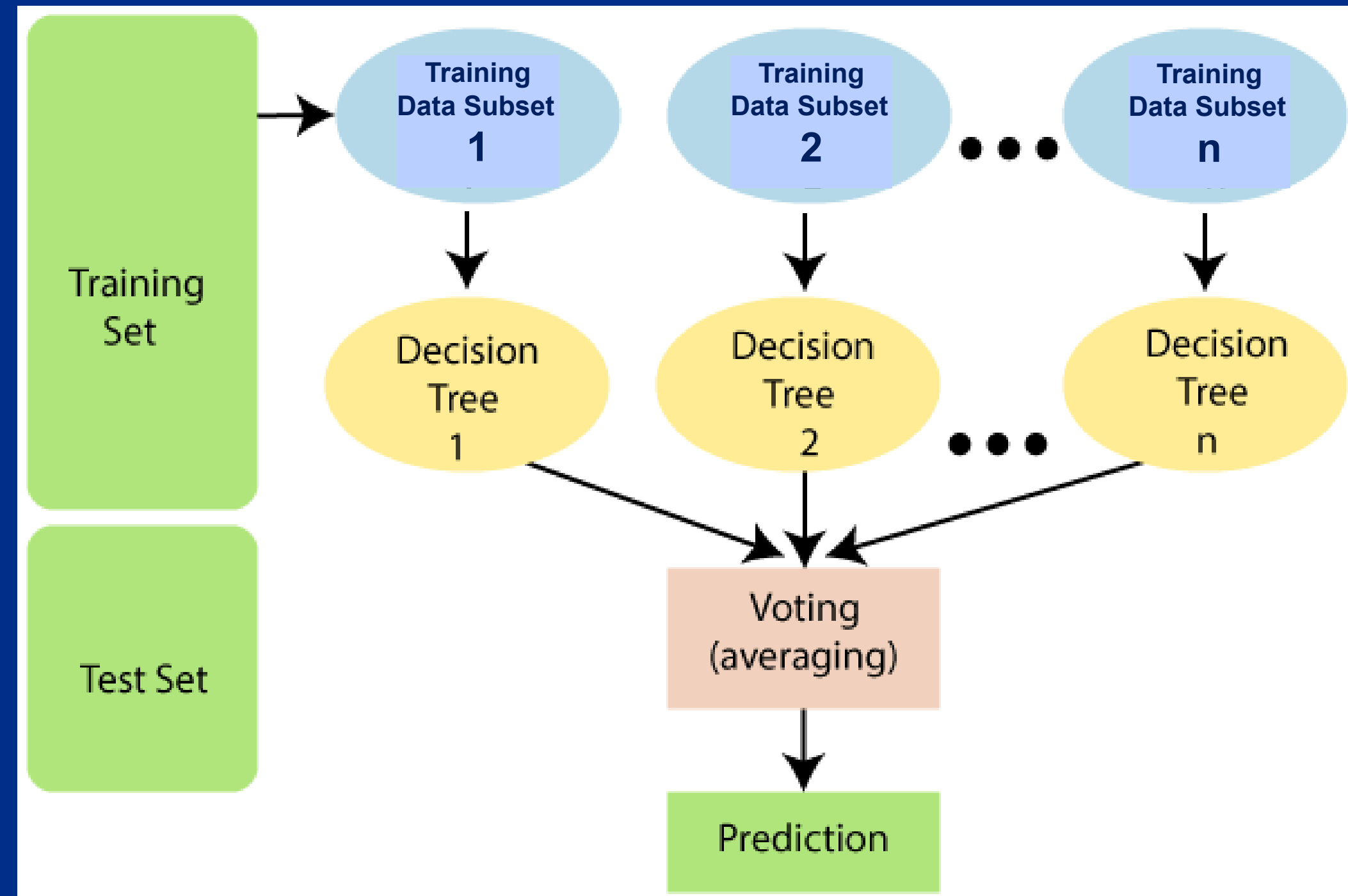
Random forest is a supervised machine learning model and it can be used for both Classification and Regression problems.

It is based on the concept of ensemble learning, which is a process of combining multiple classifiers to solve a complex problem and to improve the performance of the model.

Random Forest is a classifier that contains a number of decision trees on various subsets of the given dataset and takes the average to improve the predictive accuracy of that dataset.

It is possible that some of the trees may predict the correct output, while others may not. But together, all the trees predict the correct output.

The greater number of trees in the forest leads to higher accuracy and prevents the problem of overfitting.



Assumptions for a better Random forest classifier:

There should be some actual values in the feature variable of the dataset so that the classifier can predict accurate results rather than a guessed result.

The predictions from each tree must have very low correlations

Advantages of Random Forest

- Random Forest is capable of performing both Classification and Regression tasks.
- It is capable of handling large datasets with high dimensionality.
- It enhances the accuracy of the model and prevents the overfitting issue.
- It takes less training time compared to other algorithms
- It can also maintain accuracy when a large proportion of data is missing.

Disadvantages of Random Forest

- Although random forest can be used for both classification and regression tasks, it is not more suitable for Regression tasks.

Building Random Forest:

Step-1: Select 'K' data points randomly (Subset) from the training set (facilities random feature selection).

Step-2: Build the decision trees based on those data points.

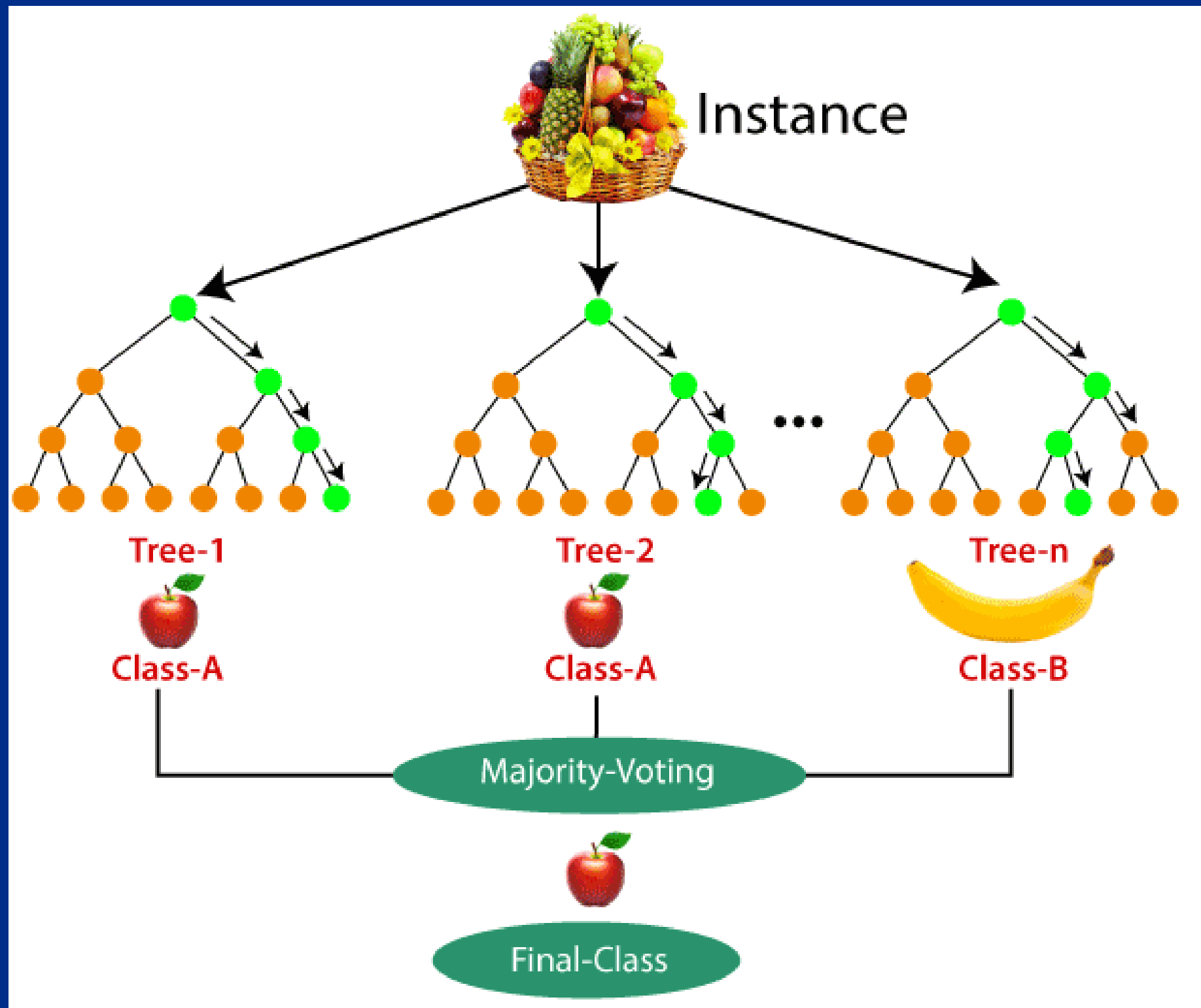
Step-3: Choose the number N for decision trees that you want to build.

Step-4: Repeat Step 1 & 2 until N number of trees are created.

Step-5: For new data points, find the predictions of each decision tree, and assign the new data points to the category that wins the majority votes.

Example: Suppose there is a dataset that contains multiple fruit images. So, this dataset is given to the Random forest classifier.

The dataset is divided into subsets and given to each decision tree. During the training phase, each decision tree produces a prediction result, and when a new data point occurs, then based on the majority of results, the Random Forest classifier predicts the final decision.



Applications of Random Forest:

Banking: Banking sector mostly uses this algorithm for the identification of loan risk.

Medicine: With the help of this algorithm, disease trends and risks of the disease can be identified.

Land Use: We can identify the areas of similar land use by this algorithm.

Marketing: Marketing trends can be identified using this algorithm.

Key Features of Random Forest:

- i) High Predictive Accuracy:** Imagine Random Forest as a team of decision-making wizards. Each wizard (decision tree) looks at a part of the problem, and together, they weave their insights into a powerful prediction tapestry. This teamwork often results in a more accurate model than what a single wizard could achieve.
- ii) Resistance to Overfitting:** Random Forest is like a cool-headed mentor guiding its apprentices (decision trees). Instead of letting each apprentice memorize every detail of their training, it encourages a more well-rounded understanding. This approach helps prevent getting too caught up with the training data which makes the model less prone to overfitting.
- iii) Large Datasets Handling:** Random Forest tackles large amount of data like a seasoned explorer with a team of helpers (decision trees). Each helper takes on a part of the dataset, ensuring that the expedition is not only thorough but also surprisingly quick.
- iv) Variable Importance Assessment:** Think of Random Forest as a detective at a crime scene, figuring out which clues (features) matter the most. It assesses the importance of each clue in solving the case, helping you focus on the key elements that drive predictions.

v) Built-in Cross-Validation: Random Forest is like having a personal coach that keeps you in check. As it trains each decision tree, it also sets aside a secret group of cases (out-of-bag) for testing. This built-in validation ensures your model doesn't just ace the training but also performs well on new challenges.

vi) Handling Missing Values: Life is full of uncertainties, just like datasets with missing values. Random Forest is the friend who adapts to the situation, making predictions using the information available. It doesn't get flustered by missing pieces; instead, it focuses on what it can confidently tell us.

vii) Parallelization for Speed: Random Forest is your time-saving buddy. Picture each decision tree as a worker tackling a piece of a puzzle simultaneously. This parallel approach taps into the power of modern tech, making the whole process faster and more efficient for handling large-scale projects.

Clustering

Clusters are groups of data objects. Clustering is an unsupervised machine learning. The effectiveness of clustering depends on how similar they are within a cluster, and how different they are than the objects in other clusters. Clustering is a set of techniques used to partition data into groups, or clusters. Some important factors that affect selecting the clusters are: the characteristics of the clusters, the features of the dataset, the number of outliers, and the number of data objects. Selecting an appropriate clustering algorithm for the given dataset is also difficult due to the number of choices available.

Three popular categories of clustering algorithms:

Partitional clustering

Hierarchical clustering

Density-based clustering

1. **Partitional clustering** divides data objects into nonoverlapping groups. In other words, no object can be a member of more than one cluster, and every cluster must have at least one object. Uses mean or centroid or medoid to represent the center of the cluster. Adapts distance-based approach to refine clusters. Effective for medium or small size datasets

2. **Hierarchical clustering** determines cluster assignments by building a hierarchy or a tree. This is implemented by either a bottom-up or a top-down approach:

a) **Agglomerative clustering** is the bottom-up approach. It merges the two points that are the most similar until all points have been merged into a single cluster.

b) **Divisive clustering** is the top-down approach. It starts with all points as one cluster and splits the least similar clusters at each step until only single data points remain.

3. **Density-based clustering** determines cluster assignments based on the density of data points in a region. Clusters are assigned where there are high densities of data points separated by low-density regions. May filter out outliers.

K-Means Clustering

K-Means Clustering is an Unsupervised Machine Learning algorithm, which groups the unlabeled dataset into different clusters.

K-means clustering, assigns data points to one of the K clusters depending on their distance from the center of the clusters.

K-means is essentially a grouping of things based on how similar or different they are to one another.

We are given a data set of items, with certain features, and values for these features (like a vector). The task is to categorize those items into groups.

Steps:

1. Let the number of clusters be 'k'
2. Randomly initialize 'k' values (called mean or centroid of the cluster).
3. Assign each data point to one of the cluster based on its distance from centroid of the cluster.
4. After assigning each point to one of the cluster, the mean value or centroid for each cluster is computed.
5. Repeat steps 2 to 4, until all points find good cluster.

Number of Clusters (k):

Number of clusters are guessed or predefined at the starting of clustering process. Choosing optimal number of clusters is a big task.

An intuitive method to initialize the mean for each cluster is to choose random items in the data set.

Another method is to initialize the means is to choose random values between the boundaries of the data set i.e. between the minimum and maximum values

Two methods are used to choose appropriate or optimum number of clusters:

- i) The elbow method :
- ii) The silhouette coefficient:

i) The elbow method :

The quality of the cluster assignments is determined by computing the total **Sum of the Squared Error (SSE)** of all clusters. If the first cluster has N_1 number of points (data) with centroid c_1 , and if any point in that cluster is denoted as 'n', then the distance of each point from the centroid is calculated and squared. The sum of all these squared values are computed as given below.

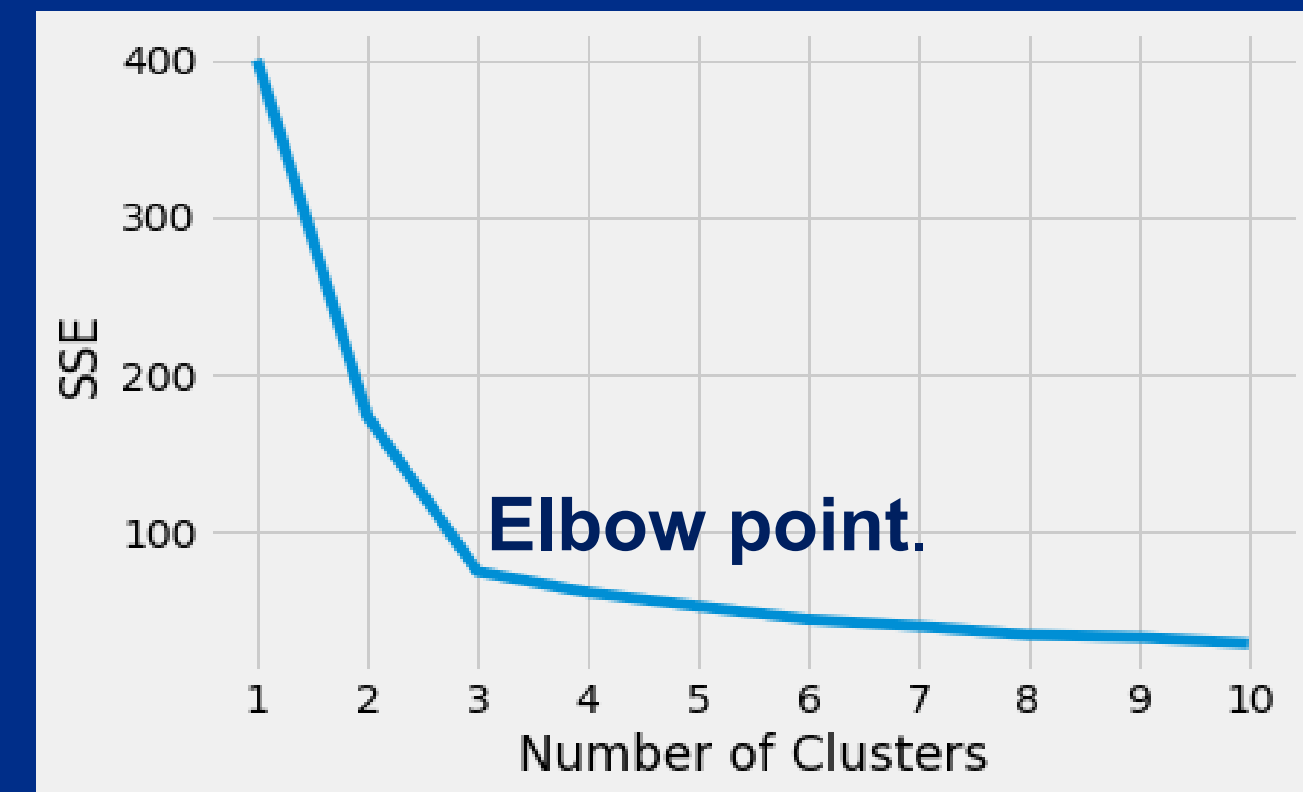
$$\sum_n^{N_1} \{distance(n, c_1)\}^2$$

If there are 'k' number of clusters then

$$\mathbf{SSE} = \sum_n^{N_1} \{distance(n, c_1)\}^2 + \sum_n^{N_2} \{distance(n, c_2)\}^2 \dots \sum_n^{N_k} \{distance(n, c_k)\}^2$$

Since this is a measure of error, the objective of k-means is to try to minimize this value.

To perform the elbow method, run several k-means, increment k with each iteration, and record the SSE. When you plot SSE as a function of the number of clusters, notice that SSE continues to decrease as you increase k. As more centroids are added, the distance from each point to its closest centroid will decrease. There's a sweet spot where the SSE curve starts to bend known as the **elbow point**.

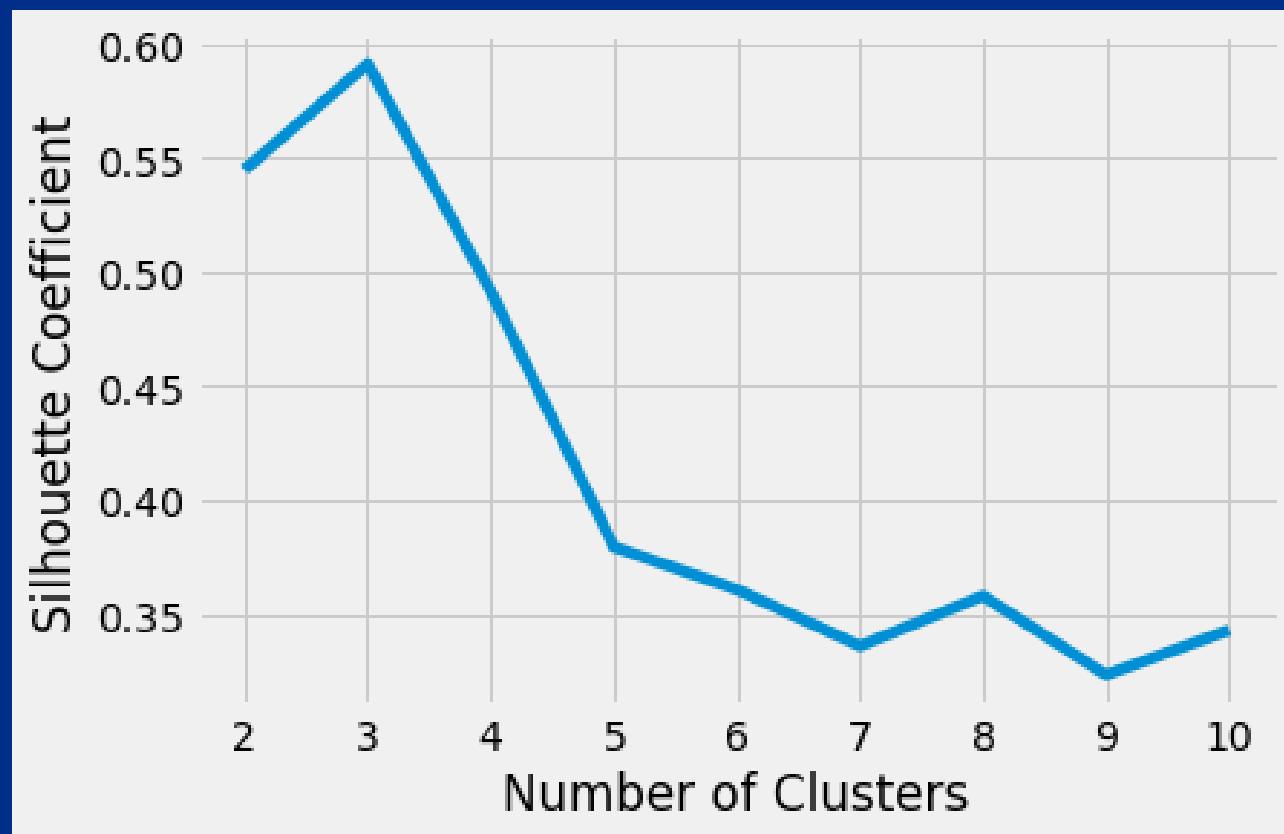


ii) The silhouette coefficient:

The silhouette coefficient is a measure of cluster cohesion and separation. It quantifies how well a data point fits into its assigned cluster based on two factors:

- a) How close the data point is to other points in the cluster
- b) How far away the data point is from points in other clusters

Silhouette coefficient values range between -1 and 1. Larger numbers indicate that samples are closer to their clusters than they are to other clusters.



Ultimately, your decision on the number of clusters to use should be guided by a combination of domain knowledge and clustering evaluation metrics.

Strengths:

Less complex

Flexible

Performance and efficiency is very high

Weakness:

Number of clusters are guessed or predefined at the starting of clustering process.

Involves randomness, so some times may not provide optimal clusters

Advantages:

K-Means is faster as compared to other clustering technique.

K-Means clustering performs best when data are well separated.

It provides strong coupling between the data points.

Disadvantages:

When data points overlapped this clustering is not suitable.

K-Means cluster do not provide clear information regarding the quality of clusters.

Different initial assignment of cluster centroid may lead to different clusters.

K-Means algorithm is sensitive to noise.

It may stuck in local minima.

Problem 1:

Apply k-means clustering algorithm to group the following data points into 2 clusters. Initial centroids will be the farthest ie a pair of data points with maximum Euclidean distance

Subject	A	B
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5

The Euclidean distance between two points (x1,y1) and (x2,y2) is given by:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

We compute distances between all pairs:

Point 1	Point 2	Distance
(1.0, 1.0)	(1.5, 2.0)	$\sqrt{(1.5 - 1.0)^2 + (2.0 - 1.0)^2} = \sqrt{0.25 + 1} = \sqrt{1.25} = 1.12$
(1.0, 1.0)	(3.0, 4.0)	$\sqrt{(3.0 - 1.0)^2 + (4.0 - 1.0)^2} = \sqrt{4 + 9} = \sqrt{13} = 3.61$
(1.0, 1.0)	(5.0, 7.0)	$\sqrt{(5.0 - 1.0)^2 + (7.0 - 1.0)^2} = \sqrt{16 + 36} = \sqrt{52} = 7.21$
(1.0, 1.0)	(3.5, 5.0)	$\sqrt{(3.5 - 1.0)^2 + (5.0 - 1.0)^2} = \sqrt{6.25 + 16} = \sqrt{22.25} = 4.71$
(1.0, 1.0)	(4.5, 5.0)	$\sqrt{(4.5 - 1.0)^2 + (5.0 - 1.0)^2} = \sqrt{12.25 + 16} = \sqrt{28.25} = 5.32$
(1.0, 1.0)	(3.5, 4.5)	$\sqrt{(3.5 - 1.0)^2 + (4.5 - 1.0)^2} = \sqrt{6.25 + 12.25} = \sqrt{18.5} = 4.30$
(1.5, 2.0)	(3.0, 4.0)	$\sqrt{(3.0 - 1.5)^2 + (4.0 - 2.0)^2} = \sqrt{2.25 + 4} = \sqrt{6.25} = 2.5$
(1.5, 2.0)	(5.0, 7.0)	$\sqrt{(5.0 - 1.5)^2 + (7.0 - 2.0)^2} = \sqrt{12.25 + 25} = \sqrt{37.25} = 6.11$
(3.0, 4.0)	(5.0, 7.0)	$\sqrt{(5.0 - 3.0)^2 + (7.0 - 4.0)^2} = \sqrt{4 + 9} = \sqrt{13} = 3.61$
(3.5, 5.0)	(4.5, 5.0)	$\sqrt{(4.5 - 3.5)^2 + (5.0 - 5.0)^2} = \sqrt{1 + 0} = 1.0$
(3.5, 5.0)	(3.5, 4.5)	$\sqrt{(3.5 - 3.5)^2 + (4.5 - 5.0)^2} = \sqrt{0 + 0.25} = 0.5$

Subject	A	B
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5

The farthest pair: (1.0,1.0) and (5.0,7.0) with a distance of 7.21.

These will be the initial centroids.

Step 2: Assign Each Point to the Nearest Centroid

Using the two centroids (1.0, 1.0) and (5.0, 7.0), we calculate the distance of each point to both centroids.

Distance to Centroid (1.0,1.0)

Point	Distance
(1.0, 1.0)	0
(1.5, 2.0)	1.12
(3.0, 4.0)	3.61
(5.0, 7.0)	7.21
(3.5, 5.0)	4.71
(4.5, 5.0)	5.32
(3.5, 4.5)	4.30

Distance to Centroid (5.0,7.0)

Point	Distance
(1.0, 1.0)	7.21
(1.5, 2.0)	6.11
(3.0, 4.0)	3.61
(5.0, 7.0)	0
(3.5, 5.0)	2.5
(4.5, 5.0)	2.24
(3.5, 4.5)	2.92

Assign each point to the nearest centroid:

Cluster 0 (Closer to (1.0, 1.0)): (1.0, 1.0), (1.5, 2.0)

Cluster 1 (Closer to (5.0, 7.0)): (3.0, 4.0), (5.0, 7.0), (3.5, 5.0), (4.5, 5.0), (3.5, 4.5)

Step 3: Compute New Centroids

Assign each point to the nearest centroid:

Cluster 0 (Closer to (1.0, 1.0)): (1.0, 1.0), (1.5, 2.0)

Cluster 1 (Closer to (5.0, 7.0)): (3.0, 4.0), (5.0, 7.0), (3.5, 5.0), (4.5, 5.0), (3.5, 4.5)

Centroid formula:

$$C = \frac{1}{N} \sum X_i$$

New Centroid for Cluster 0:

$$C_0 = \left(\frac{1.0 + 1.5}{2}, \frac{1.0 + 2.0}{2} \right) = (1.25, 1.5)$$

New Centroid for Cluster 1:

$$\begin{aligned} C_1 &= \left(\frac{3.0 + 5.0 + 3.5 + 4.5 + 3.5}{5}, \frac{4.0 + 7.0 + 5.0 + 5.0 + 4.5}{5} \right) \\ &= \left(\frac{19.5}{5}, \frac{25.5}{5} \right) = (3.9, 5.1) \end{aligned}$$

Step 4: Check for Convergence

Since the new centroids (1.25, 1.5) and (3.9, 5.1) are different from the initial centroids, we repeat Step 2 and Step 3 until they stop changing.

Final clusters after convergence:

- Cluster 0: (1.0, 1.0), (1.5, 2.0)
- Cluster 1: (3.0, 4.0), (5.0, 7.0), (3.5, 5.0), (4.5, 5.0), (3.5, 4.5)

Final centroids:

- Cluster 0 centroid: (1.25, 1.5)
- Cluster 1 centroid: (3.9, 5.1)

THANK YOU

THANK YOU